```
// ========== drizzle.config.ts ==========
import { defineConfig } from 'drizzle-kit';

export default defineConfig({
  schema: './src/storage/database/shared/schema.ts',
  out: './drizzle',
  dialect: 'postgresql',
  dbCredentials: {
    url: process.env.DATABASE_URL!,
  },
});

// ========== next.config.ts ==========
import type { NextConfig } from 'next';

const supabaseHost = process.env.COZE_SUPABASE_URL
  ? new URL(process.env.COZE_SUPABASE_URL).hostname
  : '*.supabase.co';

const cozeDomain = process.env.COZE_PROJECT_DOMAIN_DEFAULT
  ? new URL('https://' + process.env.COZE_PROJECT_DOMAIN_DEFAULT).hostname
  : '*.dev.coze.site';

const nextConfig: NextConfig = {
  allowedDevOrigins: ['*.dev.coze.site'],
  env: {
    NEXT_PUBLIC_SUPABASE_URL: process.env.COZE_SUPABASE_URL || '',
    NEXT_PUBLIC_SUPABASE_ANON_KEY: process.env.COZE_SUPABASE_ANON_KEY || '',
  },
  // ????
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: '*', pathname: '/**' },
    ],
    formats: ['image/avif', 'image/webp'],
  },
  // ??
  compress: true,
  // ????
  poweredByHeader: false,
  reactStrictMode: false, // ???????double render???????
  // ?????
  experimental: {
    optimizePackageImports: [
      'recharts',
      'lucide-react',
      '@radix-ui/react-icons',
    ],
  },
  async headers() {
```

```
  return [
    {
      source: '/(.*)',
      headers: [
        { key: 'X-Frame-Options', value: 'DENY' },
        { key: 'X-Content-Type-Options', value: 'nosniff' },
        { key: 'X-XSS-Protection', value: '1; mode=block' },
        { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
        {
          key: 'Content-Security-Policy',
          value: [
            "default-src 'self'",
            `script-src 'self' 'unsafe-inline' 'unsafe-eval' https://${cozeDomain}`,
            `style-src 'self' 'unsafe-inline' https://fonts.googleapis.cn`,
            `font-src 'self' https://fonts.gstatic.cn`,
            `img-src 'self' data: blob: https:`,
            `connect-src 'self' https://${supabaseHost} https://${cozeDomain} https://api.coze.cn ...
            `frame-ancestors 'none'`,
            "base-uri 'self'",
            "form-action 'self'",
          ].join('; '),
        },
        {
          key: 'Permissions-Policy',
          value: 'camera=(), microphone=(), geolocation=()',
        },
      ],
    },
    // ????????
    {
      source: '/_next/static/(.*)',
      headers: [
        { key: 'Cache-Control', value: 'public, max-age=31536000, immutable' },
      ],
    },
    // API???
    {
      source: '/api/dashboard',
      headers: [
        { key: 'Cache-Control', value: 'private, max-age=30, stale-while-revalidate=60' },
      ],
    },
    {
      source: '/api/auth/profile',
      headers: [
        { key: 'Cache-Control', value: 'private, max-age=10, stale-while-revalidate=30' },
      ],
    },
  ];
},
```

```
};

export default nextConfig;

// ========== src/app/(auth)/forgot-password/page.tsx ==========
'use client';

import React, { useState } from 'react';
import Link from 'next/link';
import { Zap, Loader2, ArrowLeft, Mail, KeyRound } from 'lucide-react';
import { getSupabaseBrowser } from '@/lib/supabase-browser';

export default function ForgotPasswordPage() {
  const [email, setEmail] = useState('');
  const [loading, setLoading] = useState(false);
  const [sent, setSent] = useState(false);
  const [error, setError] = useState('');

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      const sb = await getSupabaseBrowser();
      const domain = typeof window !== 'undefined' ? window.location.origin : '';
      const { error: authError } = await sb.auth.resetPasswordForEmail(email, {
        redirectTo: `${domain}/login?reset=1`,
      });

      if (authError) {
        if (authError.message.includes('rate limit')) {
          setError('????????????');
        } else {
          setError(authError.message);
        }
        return;
      }

      setSent(true);
    } catch {
      setError('????????');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-blue-50 via...
      <div className="w-full max-w-md">
```

```
<div className="text-center mb-8">
  <div className="inline-flex items-center justify-center w-16 h-16 rounded-2xl bg-gradient-...
    <Zap className="w-8 h-8 text-white" />
  </div>
  <h1 className="text-2xl font-bold text-gray-900">????</h1>
  <p className="text-lg text-gray-700 mt-1 font-medium">????</p>
</div>

<div className="bg-white rounded-2xl shadow-xl shadow-gray-200/50 border border-gray-100 p-8...
  {sent ? (
    <div className="text-center">
      <div className="mx-auto w-20 h-20 rounded-full bg-green-100 flex items-center justify-...
        <Mail className="w-10 h-10 text-green-600" />
      </div>
      <h2 className="text-xl font-bold text-gray-900 mb-3">???????</h2>
      <p className="text-base text-gray-600 mb-2">
        ????????
      </p>
      <p className="text-lg font-semibold text-gray-900 mb-6">{email}</p>
      <p className="text-base text-gray-500 mb-8 leading-relaxed">
        ????????????????????????????????
      </p>
      <Link
        href="/login"
        className="inline-flex items-center gap-2 px-8 py-3 rounded-xl bg-gradient-to-r from...
      >
        <ArrowLeft className="w-5 h-5" />
        ????
      </Link>
    </div>
  ) : (
    <>
      <div className="flex items-center gap-3 mb-2">
        <div className="w-10 h-10 rounded-xl bg-orange-100 flex items-center justify-center"...
          <KeyRound className="w-5 h-5 text-orange-600" />
        </div>
        <h2 className="text-xl font-bold text-gray-900">?????</h2>
      </div>
      <p className="text-base text-gray-500 mb-6 leading-relaxed">
        ??????????????????????
      </p>

      {error && (
        <div className="mb-6 p-4 rounded-xl bg-red-50 border border-red-200 text-red-700 tex...
          {error}
        </div>
      )}

      <form onSubmit={handleSubmit} className="space-y-6">
        <div>
```

```
                <label className="block text-base font-semibold text-gray-700 mb-2">????</label>
                <input
                  type="email"
                  value={email}
                  onChange={(e) => setEmail(e.target.value)}
                  placeholder="???????????"
                  required
                  className="w-full px-4 py-3.5 rounded-xl border-2 border-gray-200 focus:border-o...
                />
              </div>

              <button
                type="submit"
                disabled={loading}
                className="w-full py-3.5 rounded-xl bg-gradient-to-r from-orange-500 to-orange-600...
              >
                {loading ? <Loader2 className="w-5 h-5 animate-spin" /> : <Mail className="w-5 h-5...
                {loading ? '???...' : '??????'}
              </button>
            </form>

            <div className="mt-6 text-center">
              <Link
                href="/login"
                className="inline-flex items-center gap-2 text-base text-orange-500 hover:text-ora...
              >
                <ArrowLeft className="w-4 h-4" />
                ????
              </Link>
            </div>
          </>
        )}
      </div>
    </div>
  </div>
  );
}

// ========== src/app/(auth)/intro-efficiency/page.tsx ==========
import Link from 'next/link';
import {
  Zap, Bot, ClipboardCheck, FileText, BookOpen,
  BarChart3, ShieldCheck, ArrowRight, CheckCircle2,
  Flame, Lock, Star, TrendingUp, Award,
} from 'lucide-react';

export default function IntroEfficiencyPage() {
  return (
    <div className="min-h-screen bg-gradient-to-b from-slate-950 via-blue-950 to-slate-900 text-whit...
      {/* ?? Header ?? */}
```

```
<header className="sticky top-0 z-50 bg-slate-950/80 backdrop-blur border-b border-white/10">
  <div className="max-w-5xl mx-auto px-6 h-14 flex items-center justify-between">
    <span className="text-lg font-bold tracking-wide">????</span>
    <div className="flex items-center gap-4">
      <Link href="/login" className="text-sm text-white/70 hover:text-white transition">??</Li...
      <Link
        href="/register?role=efficiency_user"
        className="text-sm px-4 py-1.5 bg-cyan-500 hover:bg-cyan-400 text-blue-950 font-semibo...
      >
        ????
      </Link>
    </div>
  </div>
</header>

{/* ?? Hero: 99??? ?? */}
<section className="max-w-4xl mx-auto px-6 pt-20 pb-16 text-center">
  <div className="inline-flex items-center gap-2 px-4 py-1.5 bg-cyan-500/10 border border-cyan...
    <Zap className="w-4 h-4 text-cyan-400" />
    <span className="text-cyan-300 text-sm font-medium">99??? ? ????????</span>
  </div>
  <h1 className="text-4xl md:text-5xl font-extrabold leading-tight mb-6">
    ?????<span className="text-cyan-400">?</span><br />
    ???AI<span className="text-cyan-400">??</span>
  </h1>
  <p className="text-lg text-white/60 max-w-2xl mx-auto mb-10">
    99?/??AI??????+3?AI??+?????+????+???
  </p>
  <Link
    href="/register?role=efficiency_user"
    className="inline-flex items-center gap-2 px-8 py-3.5 bg-cyan-500 hover:bg-cyan-400 text-b...
  >
    ??????? 99?/?
    <ArrowRight className="w-5 h-5" />
  </Link>
</section>

{/* ?? ???????? ?? */}
<section className="max-w-4xl mx-auto px-6 pb-20">
  <h2 className="text-2xl font-bold text-center mb-10">?????????</h2>
  <div className="grid gap-6 md:grid-cols-3">
    {[
      {
        icon: ShieldCheck,
        title: '??????????????',
        desc: '99?/?????????????????????',
      },
      {
        icon: Zap,
        title: '??????980',
```

```
        desc: '??????????????????????????????',
      },
      {
        icon: CheckCircle2,
        title: '???????????????',
        desc: '???????????AI???????AI???????',
      },
    ].map((item) => (
      <div
        key={item.title}
        className="bg-white/5 border border-white/10 rounded-2xl p-6 hover:border-cyan-400/30 ...
      >
        <item.icon className="w-8 h-8 text-cyan-400 mb-4" />
        <p className="font-bold text-white mb-2">? {item.title}</p>
        <p className="text-sm text-white/50">{item.desc}</p>
      </div>
    ))}
  </div>
</section>

{/* ?? ????????? ?? */}
<section className="max-w-4xl mx-auto px-6 pb-20">
  <h2 className="text-2xl font-bold text-center mb-3">?????????</h2>
  <p className="text-center text-white/50 mb-10">???????AI??3???</p>
  <div className="grid gap-5 md:grid-cols-2">
    {[
      {
        emoji: '?',
        pain: '?????????????',
        solution: 'AI???3??????',
        icon: Bot,
      },
      {
        emoji: '?',
        pain: '????????????',
        solution: 'AI???????',
        icon: ClipboardCheck,
      },
      {
        emoji: '?',
        pain: '?????????',
        solution: 'SOP??????????',
        icon: FileText,
      },
      {
        emoji: '?',
        pain: '?????????',
        solution: '???????????',
        icon: BookOpen,
      },
```

```
    ].map((item) => (
      <div
        key={item.pain}
        className="bg-white/5 border border-white/10 rounded-2xl p-5 flex gap-4 items-start ho...
      >
        <div className="flex-shrink-0 w-12 h-12 rounded-xl bg-cyan-500/10 flex items-center ju...
          <item.icon className="w-6 h-6 text-cyan-400" />
        </div>
        <div>
          <p className="text-white/70 text-sm mb-1">{item.emoji} {item.pain}</p>
          <p className="font-semibold text-cyan-300">? {item.solution}</p>
        </div>
      </div>
    ))}
  </div>
  <p className="text-center mt-8 text-lg font-semibold text-white/80">
    ?????????AI??
  </p>
</section>

{/* ?? ????980??? ?? */}
<section className="max-w-4xl mx-auto px-6 pb-20">
  <div className="bg-gradient-to-br from-amber-500/10 to-orange-500/5 border border-amber-400/...
    <div className="flex items-center gap-3 mb-4">
      <TrendingUp className="w-7 h-7 text-amber-400" />
      <h2 className="text-2xl font-bold">980????</h2>
    </div>
    <p className="text-white/70 mb-4 text-lg">
      <span className="text-cyan-400 font-semibold">99?????</span>?<span className="text-amber...
    </p>
    <p className="text-white/50 mb-6">
      ???????????????
    </p>
    <div className="grid gap-3 md:grid-cols-2">
      {[
        { icon: BookOpen, text: '25?????' },
        { icon: BarChart3, text: 'KPI???' },
        { icon: ClipboardCheck, text: 'AI???5?' },
        { icon: Award, text: '??????' },
      ].map((item) => (
        <div key={item.text} className="flex items-center gap-3 text-white/60">
          <item.icon className="w-5 h-5 text-amber-400" />
          <span>{item.text}</span>
        </div>
      ))}
    </div>
    <p className="mt-6 text-sm text-amber-300/70">
      ??????????980????????99????????881? ?
    </p>
  </div>
```

```
</section>

{/* ?? ????????? ?? */}
<section className="max-w-4xl mx-auto px-6 pb-20">
  <h2 className="text-2xl font-bold text-center mb-10">99 vs 980 ????</h2>
  <div className="overflow-x-auto">
    <table className="w-full text-sm border-collapse">
      <thead>
        <tr className="border-b border-white/10">
          <th className="text-left py-3 px-4 text-white/50 font-medium">??</th>
          <th className="text-center py-3 px-4 font-semibold text-cyan-400">
            ??? 99/?
          </th>
          <th className="text-center py-3 px-4 font-semibold text-amber-400">
            ??? 980
          </th>
        </tr>
      </thead>
      <tbody>
        {[
          { name: 'AI????????', free: true, pro: true },
          { name: 'AI?? - ????', free: true, pro: true },
          { name: 'AI?? - SOP??', free: true, pro: true },
          { name: 'AI?? - ????', free: true, pro: true },
          { name: 'AI?? - ????', free: false, pro: true },
          { name: 'AI?? - ????', free: false, pro: true },
          { name: '?????', free: true, pro: true },
          { name: '????', free: true, pro: true },
          { name: '???', free: true, pro: true },
          { name: '25?????', free: false, pro: true },
          { name: 'KPI???', free: false, pro: true },
          { name: '??????', free: false, pro: true },
          { name: '?????', free: false, pro: true },
        ].map((row) => (
          <tr key={row.name} className="border-b border-white/5 hover:bg-white/5 transition">
            <td className="py-3 px-4 text-white/70">{row.name}</td>
            <td className="py-3 px-4 text-center">
              {row.free ? (
                <CheckCircle2 className="w-5 h-5 text-cyan-400 mx-auto" />
              ) : (
                <Lock className="w-4 h-4 text-white/20 mx-auto" />
              )}
            </td>
            <td className="py-3 px-4 text-center">
              <CheckCircle2 className="w-5 h-5 text-amber-400 mx-auto" />
            </td>
          </tr>
        ))}
      </tbody>
    </table>
```

```
  </div>
</section>

{/* ?? ?????????? ?? */}
<section className="max-w-4xl mx-auto px-6 pb-20">
  <div className="bg-white/5 border border-white/10 border-dashed rounded-2xl p-10 text-center...
    <Star className="w-8 h-8 text-white/20 mx-auto mb-4" />
    <p className="text-white/30 text-lg font-medium mb-2">?????????</p>
    <p className="text-white/20 text-sm">????</p>
  </div>
</section>

{/* ?? ??CTA ?? */}
<section className="max-w-4xl mx-auto px-6 pb-10 text-center">
  <h2 className="text-3xl font-extrabold mb-4">???AI??</h2>
  <p className="text-white/50 mb-8">99?/? ? ????? ? ?????</p>
  <Link
    href="/register?role=efficiency_user"
    className="inline-flex items-center gap-2 px-10 py-4 bg-cyan-500 hover:bg-cyan-400 text-bl...
  >
    ??????? 99?/?
    <ArrowRight className="w-5 h-5" />
  </Link>
</section>

{/* ?? ???? ?? */}
<section className="max-w-4xl mx-auto px-6 pb-16">
  <h3 className="text-center text-white/50 text-sm mb-6">??????<a href="/contact" className="u...
  <div className="grid md:grid-cols-2 gap-4">
    {[
      { name: '???', desc: '??AI????', href: '/register?role=efficiency_user', active: true },
      { name: '???', desc: '???????', href: '/intro/personal' },
      { name: '???', desc: '???????', href: '/intro/professional' },
      { name: '???', desc: '???????', href: '/intro/flagship' },
    ].map((v) => (
      <Link
        key={v.name}
        href={v.href}
        className="block rounded-xl border border-white/10 bg-white/5 hover:border-white/20 p-...
      >
        <p className={`font-bold text-lg ${v.active ? 'text-cyan-400' : 'text-white/80'}`}>
          {v.name} {v.active && '?'}
        </p>
        <p className="text-white/40 text-xs mt-1">{v.desc}</p>
      </Link>
    ))}
  </div>
  <div className="text-center mt-6">
    <Link href="/login" className="text-white/30 hover:text-white/50 text-sm transition">
      ? ????
```

```
          </Link>
        </div>
      </section>

      {/* ?? Footer ?? */}
      <footer className="border-t border-white/10 py-8 text-center text-white/30 text-xs">
        <p>? 2025 ???? ? 1051202571@qq.com</p>
        <p className="mt-1">???????????????</p>
      </footer>
    </div>
  );
}

// ========== src/app/(auth)/layout.tsx ==========
export default function AuthLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return <>{children}</>;
}

// ========== src/app/(auth)/login/page.tsx ==========
'use client';

import React, { useState, useEffect } from 'react';
import { getSupabaseBrowser } from '@/lib/supabase-browser';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
import { Zap, Eye, EyeOff, Loader2, CheckCircle2, ArrowRight } from 'lucide-react';
import { validateEmail, validatePassword } from '@/lib/validate';
import Captcha from '@/components/captcha';

export default function LoginPage() {
  const router = useRouter();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [agreed, setAgreed] = useState(false);
  const [captchaValid, setCaptchaValid] = useState(false);

  // Password reset mode
  const [resetMode, setResetMode] = useState(false);
  const [newPassword, setNewPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [resetLoading, setResetLoading] = useState(false);
  const [resetDone, setResetDone] = useState(false);
```

```
// Detect password reset callback from Supabase
useEffect(() => {
  if (typeof window !== 'undefined') {
    const hash = window.location.hash;
    if (hash.includes('type=recovery')) {
      setResetMode(true);
    }
  }
}, []);

// Redirect if already logged in
useEffect(() => {
  (async () => {
    try {
      const sb = await getSupabaseBrowser();
      const { data: { session } } = await sb.auth.getSession();
      if (session) {
        router.replace('/');
      }
    } catch { /* ignore */ }
  })();
}, [router]);

const handleResetPassword = async (e: React.FormEvent) => {
  e.preventDefault();
  setError('');
  const pwCheck = validatePassword(newPassword);
  if (!pwCheck.valid) {
    setError(pwCheck.error || '???????');
    return;
  }
  if (newPassword !== confirmPassword) {
    setError('??????????');
    return;
  }
  setResetLoading(true);
  try {
    const sb = await getSupabaseBrowser();
    const { error: updateError } = await sb.auth.updateUser({ password: newPassword });
    if (updateError) {
      setError(updateError.message || '????????');
      return;
    }
    setResetDone(true);
  } catch {
    setError('????????');
  } finally {
    setResetLoading(false);
  }
};
```

```
const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  setError('');

  // Frontend validation
  const emailCheck = validateEmail(email);
  if (!emailCheck.valid) {
    setError(emailCheck.error || '???????');
    return;
  }
  if (!password || password.length < 1) {
    setError('?????');
    return;
  }
  if (!captchaValid) {
    setError('????????');
    return;
  }
  if (!agreed) {
    setError('??????????????');
    return;
  }

  setLoading(true);

  try {
    const sb = await getSupabaseBrowser();
    const { error: authError } = await sb.auth.signInWithPassword({
      email,
      password,
    });

    if (authError) {
      const msg = authError.message;
      if (msg === 'Invalid login credentials') {
        setError('??????????????');
      } else if (msg.includes('Email not confirmed')) {
        setError('???????????????????');
      } else if (msg.includes('rate limit') || msg.includes('too many')) {
        setError('??????????5?????');
      } else if (msg.includes('??') || msg.includes('locked')) {
        setError('????????15?????');
      } else {
        setError('????: ' + msg);
      }
      return;
    }

    // Small delay to let AuthProvider process the session change
```

```
    await new Promise(r => setTimeout(r, 300));

    // Check account expiration
    try {
      const sb = await getSupabaseBrowser();
      const { data: { user: authUser } } = await sb.auth.getUser();
      if (authUser) {
        const { data: profile } = await sb
          .from('users')
          .select('user_type, role, expires_at')
          .eq('id', authUser.id)
          .maybeSingle();
        // Check if account has expired
        if (profile?.expires_at && new Date(profile.expires_at) < new Date()) {
          await sb.auth.signOut();
          setError('??????????????');
          setLoading(false);
          return;
        }
        if (profile?.role === 'admin') {
          router.push('/admin');
        } else if (profile?.user_type === 'manager') {
          router.push('/dashboard/boss-weekly');
        } else {
          router.push('/');
        }
      } else {
        router.push('/');
      }
    } catch {
      router.push('/');
    }
    router.refresh();
  } catch (err) {
    console.error('[Login] Error:', err);
    setError('????????');
  } finally {
    setLoading(false);
  }
};

return (
  <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-blue-50 via...
    <div className="w-full max-w-md">
      {/* Logo */}
      <div className="text-center mb-8">
        <div className="inline-flex items-center justify-center w-14 h-14 rounded-2xl bg-gradient-...
          <Zap className="w-8 h-8 text-white" />
        </div>
        <h1 className="text-2xl font-bold text-gray-900">????</h1>
```

```
  <p className="text-gray-700 mt-1 text-sm font-medium">????????</p>
</div>

{/* Login Card */}
<div className="bg-white rounded-2xl shadow-xl shadow-gray-200/50 border border-gray-100 p-8...
  {resetMode ? (
    resetDone ? (
      <div className="text-center">
        <div className="mx-auto w-12 h-12 rounded-full bg-green-100 flex items-center justif...
          <CheckCircle2 className="w-6 h-6 text-green-600" />
        </div>
        <h2 className="text-xl font-bold text-gray-900 mb-3">??????</h2>
        <p className="text-base text-gray-500 mb-6">????????</p>
        <button
          onClick={() => { setResetMode(false); setResetDone(false); }}
          className="w-full py-2.5 rounded-lg bg-gradient-to-r from-sky-400 to-blue-800 text...
        >
          ???
        </button>
      </div>
    ) : (
      <>
        <h2 className="text-xl font-bold text-gray-900 mb-1">?????</h2>
        <p className="text-gray-500 text-sm mb-6">????????</p>
        {error && (
          <div className="mb-4 p-4 rounded-xl bg-red-50 border border-red-200 text-red-700 t...
        )}
        <form onSubmit={handleResetPassword} className="space-y-5">
          <div>
            <label className="block text-base font-semibold text-gray-700 mb-2">???</label>
            <input type="password" value={newPassword} onChange={e => setNewPassword(e.targe...
          </div>
          <div>
            <label className="block text-base font-semibold text-gray-700 mb-2">?????</label...
            <input type="password" value={confirmPassword} onChange={e => setConfirmPassword...
          </div>
          <button type="submit" disabled={resetLoading} className="w-full py-3.5 rounded-xl ...
            {resetLoading ? <Loader2 className="w-5 h-5 animate-spin" /> : null}
            {resetLoading ? '???...' : '????'}
          </button>
        </form>
      </>
    )
  ) : (
  <>
  <h2 className="text-xl font-bold text-gray-900 mb-1">????</h2>
  <p className="text-gray-500 text-sm mb-6">??????????</p>

  {error && (
    <div className="mb-4 p-3 rounded-lg bg-red-50 border border-red-200 text-red-700 text-sm...
```

```
    {error}
  </div>
)}

<form onSubmit={handleLogin} className="space-y-6 animate-fade-in-up">
  <div>
    <label className="block text-sm font-medium text-gray-700 mb-1.5">??</label>
    <input
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      placeholder="?????"
      required
      className="w-full px-4 py-2.5 rounded-lg border border-gray-300 focus:border-sky-400...
    />
  </div>

  <div>
    <label className="block text-sm font-medium text-gray-700 mb-1.5">??</label>
    <div className="relative">
      <input
        type={showPassword ? 'text' : 'password'}
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="????"
        required
        className="w-full px-4 py-2.5 pr-10 rounded-lg border border-gray-300 focus:border...
      />
      <button
        type="button"
        onClick={() => setShowPassword(!showPassword)}
        className="absolute right-3 top-1/2 -translate-y-1/2 text-gray-400 hover:text-gray...
      >
        {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
      </button>
    </div>
  </div>

      <div className="flex items-center justify-between">
        <Link
          href="/forgot-password"
          className="text-sm text-sky-500 hover:text-sky-600 font-medium"
        >
          ?????
        </Link>
      </div>

      <div>
        <label className="block text-sm font-medium text-gray-700 mb-1.5">???</label>
        <Captcha onValidate={setCaptchaValid} />
```

```
            </div>

            {/* ????/?????? */}
            <label className="flex items-start gap-2 text-sm text-gray-600 mt-1">
              <input
                type="checkbox"
                checked={agreed}
                onChange={(e) => { setAgreed(e.target.checked); if (e.target.checked) setError('...
                className="mt-0.5 w-4 h-4 rounded border-gray-300 text-sky-500 focus:ring-sky-40...
              />
              <span>???????
                <Link href="/terms" target="_blank" className="text-[#2B7DE9] hover:underline fo...
                ?
                <Link href="/privacy" target="_blank" className="text-[#2B7DE9] hover:underline ...
              </span>
            </label>

            <button
              type="submit"
              disabled={loading}
              className="w-full py-2.5 rounded-lg bg-gradient-to-r from-sky-400 to-blue-800 text...
            >
              {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : null}
              {loading ? '???...' : '??'}
            </button>
          </form>

          <div className="mt-6 text-center text-sm text-gray-500">
            ?????
            <Link href="/register" className="text-blue-900 hover:text-blue-950 font-medium ml-1">
              ?????
            </Link>
          </div>
          <div className="mt-3 text-center">
            <Link
              href="/intro/personal"
              className="text-sm text-sky-600 hover:text-sky-700 font-medium inline-flex items-cente...
            >
              ?????? <ArrowRight className="w-3.5 h-3.5" />
            </Link>
          </div>
          </>
          )}
        </div>
      </div>
    </div>
  );
}

// ========== src/app/(auth)/privacy/page.tsx ==========
```

```
import Link from 'next/link';
import { ArrowLeft } from 'lucide-react';

export default function PrivacyPage() {
  return (
    <div className="min-h-screen bg-gray-50">
      <div className="max-w-3xl mx-auto px-4 py-8">
        <Link
          href="/"
          className="inline-flex items-center gap-1.5 text-[#2B7DE9] hover:underline mb-6 text-base"
        >
          <ArrowLeft className="w-4 h-4" />
          ??
        </Link>

        <h1 className="text-2xl font-bold text-gray-900 mb-2">????</h1>
        <p className="text-sm text-gray-500 mb-8">
          ??????????????/ ??????????<br />
          ?????2026?06?01?<br />
          ????????????????????????????????????????????????????????????????????????
        </p>

        <div className="bg-white rounded-xl shadow-sm p-6 sm:p-8 space-y-8 text-gray-800 leading-rel...
          {/* ??? */}
          <section>
            <h2 className="text-lg font-bold text-gray-900 mb-3">????????</h2>
            <div className="space-y-3">
              <p>??????"???????????"????????????????????????????????????????????????????????????????...
            </div>
          </section>

          {/* ??? */}
          <section>
            <h2 className="text-lg font-bold text-gray-900 mb-3">?????????</h2>
            <div className="space-y-3">
              <p><strong>2.1 ????</strong>?????????????????????</p>
              <ul className="list-disc pl-6 space-y-1">
                <li>?????????????????????</li>
                <li>??/??????????????????</li>
                <li>????????????????????????????????????????????????</li>
              </ul>
              <p><strong>2.2 ??????</strong>???????</p>
              <ul className="list-disc pl-6 space-y-1">
                <li>????????????</li>
                <li>?????????</li>
                <li>????????</li>
                <li>??????</li>
                <li>????</li>
                <li>???????????????????</li>
              </ul>
```

```
  </div>
</section>

{/* ??? */}
<section>
  <h2 className="text-lg font-bold text-gray-900 mb-3">????????</h2>
  <div className="space-y-3">
    <p><strong>3.1</strong> ???????????????</p>
    <ul className="list-disc pl-6 space-y-1">
      <li>?????????????????</li>
      <li>?????????????</li>
      <li>????????????</li>
    </ul>
    <p><strong>3.2</strong> ????????????</p>
    <ul className="list-disc pl-6 space-y-1">
      <li>????</li>
      <li>??????????</li>
      <li>????????????</li>
      <li>???????????????</li>
    </ul>
  </div>
</section>

{/* ??? */}
<section>
  <h2 className="text-lg font-bold text-gray-900 mb-3">??????????</h2>
  <div className="space-y-3">
    <p><strong>4.1</strong> ??????????????????????????</p>
    <p><strong>4.2</strong> ??????????????????????</p>
    <ul className="list-disc pl-6 space-y-1">
      <li>??????????</li>
      <li>?????????????????????????</li>
      <li>??????????????????????????</li>
    </ul>
  </div>
</section>

{/* ??? */}
<section>
  <h2 className="text-lg font-bold text-gray-900 mb-3">????????</h2>
  <div className="space-y-3">
    <p>?????????????????????????</p>
    <ul className="list-disc pl-6 space-y-1">
      <li>??HTTPS?????????????</li>
      <li>?????????????????????????</li>
      <li>?????Token Authentication???????????????????</li>
      <li>???????Supabase???????Supabase Inc.??????????????????????????????????????????????...
    </ul>
  </div>
</section>
```

```
{/* ??? - ??????????? */}
<section>
  <h2 className="text-lg font-bold text-gray-900 mb-3">??????????</h2>
  <div className="space-y-3">
    <p><strong>6.1 ?????</strong>??????Supabase Inc.????????????????????????????????Supab...
    <p><strong>6.2 ???????</strong>????????????????????????????????????????????????????...
    <p><strong>6.3 ???????</strong>??????????Supabase Inc.?supabase.com?????????????????<...
    <ul className="list-disc pl-6 space-y-1">
      <li>SOC2 Type II ??????</li>
      <li>?????TLS 1.2+?????????AES-256?</li>
      <li>ISO 27001 ??????????</li>
      <li>????GDPR????</li>
    </ul>
    <p><strong>6.4 ??????</strong>????????????????????????????????????????????????...
    <p><strong>6.5 ??????</strong>????????????????????????????????????????????????...
  </div>
</section>

{/* ??? */}
<section>
  <h2 className="text-lg font-bold text-gray-900 mb-3">????????</h2>
  <div className="space-y-3">
    <p>???????????????</p>
    <ul className="list-disc pl-6 space-y-1">
      <li><strong>???</strong>?????????????????????</li>
      <li><strong>???</strong>???????????????</li>
      <li><strong>???????</strong>????????????????????????????????????????????????1...
      <li><strong>?????</strong>????????????????????????????????????????????????...
    </ul>
    <p><strong>?????????</strong>?????????????????????????????????</p>
    <ul className="list-disc pl-6 space-y-1">
      <li>??????????????????3??</li>
      <li>???????????????6???</li>
    </ul>
    <p>??????????????????????????</p>
  </div>
</section>

{/* ??? */}
<section>
  <h2 className="text-lg font-bold text-gray-900 mb-3">????????</h2>
  <div className="space-y-3">
    <p>???????????????????</p>
    <ul className="list-disc pl-6 space-y-1">
      <li>???????????????????</li>
      <li>????????/??????????????????????????????????</li>
      <li>???????????????????AI????</li>
    </ul>
  </div>
```

```
          </section>

          {/* ??? */}
          <section>
            <h2 className="text-lg font-bold text-gray-900 mb-3">????????</h2>
            <div className="space-y-3">
              <p><strong>9.1</strong> ??????????????????18????????????</p>
              <p><strong>9.2</strong> ????????????????????????????????????????????</p>
              <p><strong>9.3</strong> ????????????????????????????????????????????????????????</p>
              <p><strong>9.4</strong> ????????????????????????????????????????????????????????</p>
            </div>
          </section>

          {/* ??? - ??????7?? */}
          <section>
            <h2 className="text-lg font-bold text-gray-900 mb-3">??????</h2>
            <div className="space-y-3">
              <p>????????????????????????????????????????????????????????????????7????????????????????...
              <p>????????????????????????????????????????????????????</p>
            </div>
          </section>

          {/* ???? - ??????? */}
          <section>
            <h2 className="text-lg font-bold text-gray-900 mb-3">???????</h2>
            <div className="space-y-3">
              <p>????????????????????????????????????</p>
              <ul className="list-disc pl-6 space-y-1">
                <li>???????????</li>
                <li>?????1051202571@qq.com</li>
              </ul>
              <p>???????????15?????????????</p>
            </div>
          </section>
        </div>

        <p className="text-center text-sm text-gray-400 mt-8">
          ????????/ ??????????
        </p>
      </div>
    </div>
  );
}

// ========== src/app/(auth)/register/page.tsx ==========
'use client';

import React, { useState, useEffect, Suspense } from 'react';
import { getSupabaseBrowser } from '@/lib/supabase-browser';
import { useRouter, useSearchParams } from 'next/navigation';
```

```
import Link from 'next/link';
import { Zap, Eye, EyeOff, Loader2, ArrowLeft, ArrowRight, Users } from 'lucide-react';
import { validateEmail, validatePassword, validatePhone, validateCode } from '@/lib/validate';
import Captcha from '@/components/captcha';


export default function RegisterPage() {
  return (
    <Suspense fallback={<div className="min-h-screen flex items-center justify-center"><Loader2 clas...
      <RegisterForm />
    </Suspense>
  );
}

function RegisterForm() {
  const router = useRouter();
  const searchParams = useSearchParams();
  const inviteToken = searchParams.get('invite');
  const [step, setStep] = useState<1 | 2>(1);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  // Step 1
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);

  // Step 2
  const [role, setRole] = useState<string>(inviteToken ? 'staff' : 'efficiency_user');
  const [companyName, setCompanyName] = useState('');
  const [phone, setPhone] = useState('');

  // Invitation state
  const [inviteInfo, setInviteInfo] = useState<{ companyName: string; inviterName: string } | null>(...
  const [inviteChecked, setInviteChecked] = useState(false);
  const [agreed, setAgreed] = useState(false);
  const [redemptionCode, setRedemptionCode] = useState('');
  const [captchaValid, setCaptchaValid] = useState(false);

  // Redirect if already logged in
  useEffect(() => {
    (async () => {
      try {
        const sb = await getSupabaseBrowser();
        const { data: { session } } = await sb.auth.getSession();
        if (session) {
          router.replace('/onboarding');
        }
      } catch { /* ignore */ }
```

```
  })();
}, [router]);

// Validate invitation token
useEffect(() => {
  if (!inviteToken) { setInviteChecked(true); return; }
  (async () => {
    try {
      const res = await fetch(`/api/invitations/${inviteToken}`);
      const data = await res.json();
      if (res.ok && data.data?.status === 'pending') {
        setInviteInfo({ companyName: data.data.company_name || '??', inviterName: data.data.invite...
      } else {
        setError(data.error || '???????????');
      }
    } catch {
      setError('????????');
    }
    setInviteChecked(true);
  })();
}, [inviteToken]);

const validateStep1 = () => {
  const emailCheck = validateEmail(email);
  if (!emailCheck.valid) return emailCheck.error || '???????';
  if (!password) return '?????';
  const pwCheck = validatePassword(password);
  if (!pwCheck.valid) return pwCheck.error || '???????';
  if (password !== confirmPassword) return '???????';
  return '';
};

const handleNext = () => {
  const err = validateStep1();
  if (err) { setError(err); return; }
  setError('');
  setStep(2);
};

const handleRegister = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!captchaValid) { setError('????????'); return; }
  const codeCheck = validateCode(redemptionCode);
  if (!codeCheck.valid) { setError(codeCheck.error || '????????'); return; }
  if (phone) {
    const phoneCheck = validatePhone(phone);
    if (!phoneCheck.valid) { setError(phoneCheck.error || '????????'); return; }
  }
  setError('');
  setLoading(true);
```

```
  try {
    // 1. Register via our API (creates user record with selected role)
    const res = await fetch('/api/auth/register', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        email,
        password,
        role: role,
        phone,
        inviteToken: inviteToken || undefined,
        redemptionCode: redemptionCode.trim(),
      }),
    });

    const data = await res.json();
    if (!res.ok) {
      setError(data.error || '????');
      return;
    }

    // 2. Sign in with Supabase Auth
    const sb = await getSupabaseBrowser();
    const { error: authError } = await sb.auth.signInWithPassword({
      email,
      password,
    });

    if (authError) {
      // Registration succeeded but auto-login failed ? that's OK, redirect to login
      setError('');
      setTimeout(() => {
        router.push('/login');
      }, 500);
      return;
    }

    // Small delay to let AuthProvider process the session change
    await new Promise(r => setTimeout(r, 300));
    router.push('/onboarding');
    router.refresh();
  } catch {
    setError('????????');
  } finally {
    setLoading(false);
  }
};

return (
```

```
<div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-blue-50 via...
  <div className="w-full max-w-lg">
    {/* Logo */}
    <div className="text-center mb-6">
      <div className="inline-flex items-center justify-center w-14 h-14 rounded-2xl bg-gradient-...
        <Zap className="w-8 h-8 text-white" />
      </div>
      <h1 className="text-2xl font-bold text-gray-900">????</h1>
      <p className="text-gray-500 mt-1">????????????</p>
    </div>

    {/* Progress */}
    <div className="flex items-center gap-2 mb-6">
      <div className={`flex items-center gap-1.5 text-sm font-medium ${step >= 1 ? 'text-blue-90...
        <span className="w-6 h-6 rounded-full flex items-center justify-center text-xs font-bold...
        ????
      </div>
      <div className={`flex-1 h-0.5 rounded ${step >= 2 ? 'bg-sky-400' : 'bg-gray-200'}`} />
      <div className={`flex items-center gap-1.5 text-sm font-medium ${step >= 2 ? 'text-blue-90...
        <span className={`w-6 h-6 rounded-full flex items-center justify-center text-xs font-bol...
        ????
      </div>
    </div>

    {/* Card */}
    <div className="bg-white rounded-2xl shadow-xl shadow-gray-200/50 border border-gray-100 p-8...
      {error && (
        <div className="mb-4 p-3 rounded-lg bg-red-50 border border-red-200 text-red-700 text-sm...
          {error}
        </div>
      )}

      {step === 1 ? (
        <form onSubmit={(e) => { e.preventDefault(); handleNext(); }} className="space-y-6 anima...
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1.5">??</label>
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              placeholder="?????"
              required
              className="w-full px-4 py-2.5 rounded-lg border border-gray-300 focus:border-sky-4...
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1.5">??</label>
            <div className="relative">
              <input
```

```
          type={showPassword ? 'text' : 'password'}
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="??6?"
          required
          minLength={6}
          className="w-full px-4 py-2.5 pr-10 rounded-lg border border-gray-300 focus:bord...
        />
        <button
          type="button"
          onClick={() => setShowPassword(!showPassword)}
          className="absolute right-3 top-1/2 -translate-y-1/2 text-gray-400 hover:text-gr...
        >
          {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
        </button>
      </div>
      <p className="text-xs text-gray-500 mt-1">????8???????????</p>
    </div>

    <div>
      <label className="block text-sm font-medium text-gray-700 mb-1.5">????</label>
      <input
        type="password"
        value={confirmPassword}
        onChange={(e) => setConfirmPassword(e.target.value)}
        placeholder="??????"
        required
        className="w-full px-4 py-2.5 rounded-lg border border-gray-300 focus:border-sky-4...
      />
    </div>

    <button
      type="submit"
      className="w-full py-2.5 rounded-lg bg-gradient-to-r from-sky-400 to-blue-800 text-w...
    >
      ???
      <ArrowRight className="w-4 h-4" />
    </button>
  </form>
) : (
  <form onSubmit={handleRegister} className="space-y-5">
    {/* Invitation banner */}
    {inviteToken && inviteInfo && (
      <div className="p-4 rounded-xl bg-sky-50 border border-sky-200 mb-2">
        <div className="flex items-center gap-2 text-blue-900 font-medium mb-1">
          <Users className="w-4 h-4" />
          ????
        </div>
        <p className="text-sm text-gray-600">
          ??? <span className="font-medium text-blue-900">{inviteInfo.inviterName}</span> ...
```

```
    </p>
    <p className="text-xs text-gray-700 mt-1">??????????????????</p>
  </div>
)}

{/* ???? */}
<div className="space-y-2">
  <label className="block text-sm font-medium text-gray-700">????</label>
  <div className="grid grid-cols-2 gap-3">
    <button
      type="button"
      onClick={() => setRole('efficiency_user')}
      className={`p-3 rounded-xl border-2 text-left transition-all ${role === 'efficie...
    >
      <div className="flex items-center gap-2 mb-1">
        <span className="text-xs px-1.5 py-0.5 rounded-full bg-cyan-100 text-cyan-700 ...
        <span className="font-semibold text-sm text-gray-900">???</span>
      </div>
      <p className="text-xs text-gray-500">AI???+3???+?????+????</p>
    </button>
    <button
      type="button"
      onClick={() => setRole('personal_user')}
      className={`p-3 rounded-xl border-2 text-left transition-all ${role === 'persona...
    >
      <div className="flex items-center gap-2 mb-1">
        <span className="text-xs px-1.5 py-0.5 rounded-full bg-amber-100 text-amber-70...
        <span className="font-semibold text-sm text-gray-900">???</span>
      </div>
      <p className="text-xs text-gray-500">25????????+??AI??+KPI??</p>
      <p className="text-xs text-amber-600 mt-0.5">????????99????881?</p>
    </button>
  </div>
</div>

{/* ?????????? */}
<div className="space-y-3">
  <div className={`p-3 rounded-lg border text-sm ${role === 'efficiency_user' ? 'bg-cy...
    {role === 'efficiency_user'
      ? '????AI????????+ 3?AI?? + ????? + ???? + ??? ? ?????99??????'
      : '?????25??????? + ??AI?? + KPI??? + ?????'}
  </div>
  <div>
    <label className="block text-sm font-medium text-gray-700 mb-1.5">
      ??? <span className="text-red-500">*</span>
    </label>
    <input
      type="text"
      value={redemptionCode}
      onChange={(e) => setRedemptionCode(e.target.value.toUpperCase())}
```

```
      placeholder="??????????"
      required
      className="w-full px-4 py-2.5 rounded-lg border border-gray-300 focus:border-sky...
    />
    <p className="text-xs text-gray-500 mt-1">???????????????</p>
  </div>
</div>

{/* ???/????? */}
<div className="p-4 rounded-xl bg-blue-50 border border-blue-100 space-y-2">
  <p className="text-sm font-medium text-blue-900">??????????</p>
  <p className="text-xs text-gray-600">????????????????????????????????????????</p>
  <Link
    href="/contact"
    className="inline-flex items-center gap-1.5 text-sm font-medium text-sky-600 hover...
  >
    ?????? ?
  </Link>
</div>

<div>
  <label className="block text-sm font-medium text-gray-700 mb-1.5">??? <span classNam...
  <input
    type="tel"
    value={phone}
    onChange={(e) => setPhone(e.target.value)}
    placeholder="??????"
    className="w-full px-4 py-2.5 rounded-lg border border-gray-300 focus:border-sky-4...
  />
</div>

<label className="flex items-start gap-2 text-sm text-gray-600 mb-4 cursor-pointer sel...
  <input
    type="checkbox"
    checked={agreed}
    onChange={(e) => setAgreed(e.target.checked)}
    className="mt-0.5 w-4 h-4 rounded border-gray-300 text-sky-400 focus:ring-sky-400"
  />
  <span>
    ???????
    <Link href="/terms" target="_blank" className="text-[#2B7DE9] hover:underline font...
    ?
    <Link href="/privacy" target="_blank" className="text-[#2B7DE9] hover:underline fo...
  </span>
</label>

<div>
  <label className="block text-sm font-medium text-gray-700 mb-1.5">???</label>
  <Captcha onValidate={setCaptchaValid} />
</div>
```

```
            <div className="flex gap-3">
              <button
                type="button"
                onClick={() => setStep(1)}
                className="px-6 py-2.5 rounded-lg border border-gray-300 text-gray-700 font-medium...
              >
                <ArrowLeft className="w-4 h-4" />
                ???
              </button>
              <button
                type="submit"
                disabled={loading || !agreed || !captchaValid}
                className="flex-1 py-2.5 rounded-lg bg-gradient-to-r from-sky-400 to-blue-800 text...
              >
                {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : null}
                {loading ? '???...' : '????'}
              </button>
            </div>
          </form>
        )}

        <div className="mt-6 text-center text-sm text-gray-500">
          ?????
          <Link href="/login" className="text-blue-900 hover:text-blue-950 font-medium ml-1">
            ????
          </Link>
        </div>
        <div className="mt-3 text-center">
          <Link
            href="/intro/personal"
            className="text-sm text-sky-600 hover:text-sky-700 font-medium inline-flex items-cente...
          >
            ?????? <ArrowRight className="w-3.5 h-3.5" />
          </Link>
        </div>
      </div>
    </div>
  </div>
  );
}

// ========== src/app/(auth)/terms/page.tsx ==========
import Link from 'next/link';
import { ArrowLeft } from 'lucide-react';

export default function TermsPage() {
  return (
    <div className="min-h-screen bg-gray-50">
      <div className="max-w-3xl mx-auto px-4 py-8">
```

```
<Link
  href="/"
  className="inline-flex items-center gap-1.5 text-[#2B7DE9] hover:underline mb-6 text-base"
>
  <ArrowLeft className="w-4 h-4" />
  ??
</Link>

<h1 className="text-2xl font-bold text-gray-900 mb-2">??????</h1>
<p className="text-sm text-gray-500 mb-8">
  ??????????????/ ??????????<br />
  ?????2026?06?01?
</p>

<div className="bg-white rounded-xl shadow-sm p-6 sm:p-8 space-y-8 text-gray-800 leading-rel...
  {/* ???????? */}
  <section>
    <h2 className="text-lg font-bold text-gray-900 mb-3">??????</h2>
    <div className="space-y-3">
      <p><strong>1.1</strong> ??????????????????????????????/ ??????????????????????????????...
      <p><strong>1.2</strong> ??????????????????????????????????????????????????????????????...
      <p><strong>1.3</strong> ????????????????????????????????????????????</p>
      <p><strong>1.4 ????????</strong>??????????????????????????????????????????????????...
    </div>
  </section>

  {/* ???????????????? */}
  <section>
    <h2 className="text-lg font-bold text-gray-900 mb-3">???????????</h2>
    <div className="space-y-3">
      <p><strong>2.1 ????</strong>???????25??????????????????????????????????????????????...
      <p><strong>2.2 ????</strong>????????????????????????????????</p>
      <p><strong>2.3 ?????</strong>??????????99%????????????????????24???????</p>
      <p><strong>2.4 ????</strong>????????24????????????????????????????????24???????????</p>
    </div>
  </section>

  {/* ???????????????? */}
  <section>
    <h2 className="text-lg font-bold text-gray-900 mb-3">???????????</h2>
    <div className="space-y-3">
      <p><strong>3.1 ?????????</strong>??????????????????????</p>
      <ul className="list-disc pl-6 space-y-1">
        <li><strong>????</strong>?????????????????48?????</li>
        <li><strong>????</strong>???????????24?????</li>
        <li><strong>????</strong>??????????????????12?????</li>
      </ul>
      <p><strong>3.2 ??????</strong>????????????????????????72??????????????????????????????????...
      <p><strong>3.3 ????</strong>??????????24????????????????????????????24?????????????</p>
      <p><strong>3.4 ??????????SLA?</strong>????????????????????????</p>
```

```
// ========== ?????? ==========
// ?? 102164 ???
// ========== ?????? ==========


const DANGEROUS_TAGS_RE = /<\s*\/?\s*(script|iframe|object|embed|form|input|textarea|select|button|a...
const DANGEROUS_ATTRS_RE = /\s(on\w+|javascript:|vbscript:|data\s*:\s*text\/html)\s*=/gi;
const HTML_ENTITY_MAP: Record<string, string> = {
  '&': '&amp;',
  '<': '&lt;',
  '>': '&gt;',
  '"': '&quot;',
  "'": '&#x27;',
};

/**
 * Strip dangerous HTML tags and encode special characters.
 * Returns sanitized string or throws if exceeds maxLength.
 */
export function sanitizeString(input: unknown, maxLength: number = 1000): string {
  if (typeof input !== 'string') return '';

  let cleaned = input.trim();

  // Remove dangerous tags
  cleaned = cleaned.replace(DANGEROUS_TAGS_RE, '');

  // Remove dangerous attributes
  cleaned = cleaned.replace(DANGEROUS_ATTRS_RE, ' data-removed=');

  // Encode special characters for XSS prevention
  cleaned = cleaned.replace(/[&<>"']/g, (ch) => HTML_ENTITY_MAP[ch] || ch);

  if (cleaned.length > maxLength) {
    throw new Error(`???????????(${maxLength}?)`);
  }

  return cleaned;
}

/**
 * Sanitize for contexts where HTML is intentionally allowed (e.g. rich text).
 * Only strips script/iframe/event handlers but keeps safe HTML.
 */
export function sanitizeRichText(input: unknown, maxLength: number = 5000): string {
  if (typeof input !== 'string') return '';

  let cleaned = input.trim();
```

```
  // Remove script/iframe/embed/object tags
  cleaned = cleaned.replace(DANGEROUS_TAGS_RE, '');

  // Remove event handler attributes
  cleaned = cleaned.replace(DANGEROUS_ATTRS_RE, ' data-removed=');

  // Remove javascript: in href/src
  cleaned = cleaned.replace(/(href|src)\s*=\s*["']?\s*javascript:/gi, '$1="removed:"');

  if (cleaned.length > maxLength) {
    throw new Error(`????????????(${maxLength}?)`);
  }

  return cleaned;
}

// ??? Validation ???

const EMAIL_RE = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
const PHONE_RE = /^1[3-9]\d{9}$/;
const CODE_RE = /^[A-Za-z0-9]+$/;

export interface ValidationResult {
  valid: boolean;
  error?: string;
}

/**
 * Validate email format
 */
export function validateEmail(email: unknown): ValidationResult {
  if (typeof email !== 'string' || !email.trim()) {
    return { valid: false, error: '?????' };
  }
  const trimmed = email.trim().toLowerCase();
  if (trimmed.length > 254) {
    return { valid: false, error: '???????' };
  }
  if (!EMAIL_RE.test(trimmed)) {
    return { valid: false, error: '???????' };
  }
  return { valid: true };
}

/**
 * Validate Chinese mobile phone number
 */
export function validatePhone(phone: unknown): ValidationResult {
  if (typeof phone !== 'string' || !phone.trim()) {
    return { valid: false, error: '??????' };
```

```
  }
  const trimmed = phone.trim();
  if (!PHONE_RE.test(trimmed)) {
    return { valid: false, error: '???????????11???????' };
  }
  return { valid: true };
}

/**
 * Validate redemption code format (alphanumeric only)
 */
export function validateCode(code: unknown): ValidationResult {
  if (typeof code !== 'string' || !code.trim()) {
    return { valid: false, error: '??????' };
  }
  const trimmed = code.trim();
  if (trimmed.length < 4 || trimmed.length > 32) {
    return { valid: false, error: '????????' };
  }
  if (!CODE_RE.test(trimmed)) {
    return { valid: false, error: '???????????' };
  }
  return { valid: true };
}

/**
 * Validate password strength:
 * - Minimum 8 characters
 * - Must contain at least one letter and one number
 */
export function validatePassword(password: unknown): ValidationResult {
  if (typeof password !== 'string' || !password) {
    return { valid: false, error: '?????' };
  }
  if (password.length < 8) {
    return { valid: false, error: '????8?' };
  }
  if (!/[a-zA-Z]/.test(password)) {
    return { valid: false, error: '????????????' };
  }
  if (!/[0-9]/.test(password)) {
    return { valid: false, error: '????????????' };
  }
  if (password.length > 128) {
    return { valid: false, error: '????????128?' };
  }
  return { valid: true };
}

/**
```

```
 * Validate pagination parameters to prevent negative/oversized values
 */
export function validatePagination(params: {
  page?: unknown;
  pageSize?: unknown;
}): { page: number; pageSize: number } | ValidationResult {
  const MAX_PAGE_SIZE = 200;
  const DEFAULT_PAGE = 1;
  const DEFAULT_PAGE_SIZE = 20;

  let page = DEFAULT_PAGE;
  let pageSize = DEFAULT_PAGE_SIZE;

  if (params.page !== undefined && params.page !== null) {
    const p = Number(params.page);
    if (!Number.isFinite(p) || p < 1) {
      return { valid: false, error: '????????' };
    }
    page = Math.floor(p);
  }

  if (params.pageSize !== undefined && params.pageSize !== null) {
    const ps = Number(params.pageSize);
    if (!Number.isFinite(ps) || ps < 1) {
      return { valid: false, error: '??????????' };
    }
    pageSize = Math.min(Math.floor(ps), MAX_PAGE_SIZE);
  }

  return { page, pageSize };
}

/**
 * Validate that a value is a safe string ID (alphanumeric, dashes, underscores)
 */
export function validateId(id: unknown): ValidationResult {
  if (typeof id !== 'string' || !id.trim()) {
    return { valid: false, error: '??ID??' };
  }
  if (!/^[a-zA-Z0-9_-]+$/.test(id.trim())) {
    return { valid: false, error: 'ID?????' };
  }
  if (id.trim().length > 128) {
    return { valid: false, error: 'ID??' };
  }
  return { valid: true };
}

/**
 * Validate role value against whitelist
```

```
 */
const VALID_ROLES = ['admin', 'enterprise_admin', 'enterprise_manager', 'personal_user', 'efficiency...

export function validateRole(role: unknown): ValidationResult {
  if (typeof role !== 'string' || !role) {
    return { valid: false, error: '??????' };
  }
  if (!VALID_ROLES.includes(role)) {
    return { valid: false, error: '???????' };
  }
  return { valid: true };
}

/**
 * Batch validate multiple fields, returns first error or all-valid
 */
export function validateAll(
  checks: Array<{ name: string; result: ValidationResult }>,
): ValidationResult {
  for (const check of checks) {
    if (!check.result.valid) {
      return { valid: false, error: check.result.error };
    }
  }
  return { valid: true };
}

// ========== src/lib/version-check.ts ==========
/**
 * Lightweight version check utility for multi-tab data conflict detection.
 * Before saving, check if the server-side `updated_at` is newer than the local copy.
 */

import { toast } from 'sonner';

interface VersionedRecord {
  id: string;
  updated_at?: string | null;
}

/**
 * Check if a record has been updated since the local copy was loaded.
 * Returns true if conflict detected (server is newer), false if safe to save.
 */
export async function checkVersionConflict(
  authFetch: (url: string, init?: RequestInit) => Promise<Response>,
  apiPath: string,
  localRecord: VersionedRecord,
  localUpdatedAt: string | null | undefined,
): Promise<boolean> {
```

```
  if (!localRecord.id || !localUpdatedAt) {
    // No existing record or no timestamp, safe to proceed (new record or first save)
    return false;
  }

  try {
    const res = await authFetch(`${apiPath}?id=${localRecord.id}`);
    if (!res.ok) return false; // Can't check, allow save
    const data = await res.json();
    const serverRecord = Array.isArray(data?.data) ? data.data[0] : data?.data;
    if (!serverRecord?.updated_at) return false;

    const serverTime = new Date(serverRecord.updated_at).getTime();
    const localTime = new Date(localUpdatedAt).getTime();

    if (serverTime > localTime) {
      // Conflict detected - ask user
      return new Promise((resolve) => {
        const overwrite = confirm(
          '????????????????\n??"??"?????"??"??????'
        );
        resolve(!overwrite); // true = conflict blocking, false = user chose to overwrite
      });
    }
    return false; // No conflict
  } catch {
    // Can't check version, allow save
    return false;
  }
}

/**
 * Show a generic conflict warning toast (for batch operations where confirm is too heavy)
 */
export function showConflictToast() {
  toast.error('??????????????????');
}

// ========== src/middleware.ts ==========
import { NextRequest, NextResponse } from 'next/server';

// ??? In-memory rate limit store (single instance) ???
interface RateLimitEntry {
  count: number;
  resetAt: number; // timestamp when counter resets
}

const ipStore = new Map<string, RateLimitEntry>();
const userStore = new Map<string, RateLimitEntry>();
```

```
// Clean up expired entries every 5 minutes to prevent memory leak
setInterval(() => {
  const now = Date.now();
  for (const [key, entry] of ipStore) {
    if (now > entry.resetAt) ipStore.delete(key);
  }
  for (const [key, entry] of userStore) {
    if (now > entry.resetAt) userStore.delete(key);
  }
}, 5 * 60 * 1000);

// ??? Rate limit configs ???
interface RateLimitRule {
  windowMs: number;   // time window in milliseconds
  maxRequests: number; // max requests per window
}

const AUTH_LIMIT: RateLimitRule = { windowMs: 5 * 60 * 1000, maxRequests: 10 };       // /api/auth/*...
const REGISTER_LIMIT: RateLimitRule = { windowMs: 5 * 60 * 1000, maxRequests: 5 };    // /api/auth/r...
const AI_CHECKUP_LIMIT: RateLimitRule = { windowMs: 24 * 60 * 60 * 1000, maxRequests: 20 }; // /api/...

function getClientIP(request: NextRequest): string {
  // Try common proxy headers first
  const xff = request.headers.get('x-forwarded-for');
  if (xff) return xff.split(',')[0].trim();
  const realIp = request.headers.get('x-real-ip');
  if (realIp) return realIp.trim();
  return 'unknown';
}

function checkRateLimit(
  store: Map<string, RateLimitEntry>,
  key: string,
  windowMs: number,
  maxRequests: number,
): { allowed: boolean; remaining: number; resetAt: number } {
  const now = Date.now();
  const entry = store.get(key);

  if (!entry || now > entry.resetAt) {
    // New window
    const newEntry: RateLimitEntry = { count: 1, resetAt: now + windowMs };
    store.set(key, newEntry);
    return { allowed: true, remaining: maxRequests - 1, resetAt: newEntry.resetAt };
  }

  if (entry.count >= maxRequests) {
    return { allowed: false, remaining: 0, resetAt: entry.resetAt };
  }
```

```
  entry.count += 1;
  return { allowed: true, remaining: maxRequests - entry.count, resetAt: entry.resetAt };
}

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // ??? Rate limiting for /api/auth/register (stricter) ???
  if (pathname === '/api/auth/register') {
    const ip = getClientIP(request);
    const result = checkRateLimit(ipStore, `register:${ip}`, REGISTER_LIMIT.windowMs, REGISTER_LIMIT...
    if (!result.allowed) {
      return NextResponse.json(
        { error: '??????????5?????' },
        {
          status: 429,
          headers: {
            'Retry-After': String(Math.ceil((result.resetAt - Date.now()) / 1000)),
            'X-RateLimit-Remaining': '0',
          },
        },
      );
    }
  }

  // ??? Rate limiting for /api/auth/* (except register which has its own limit) ???
  if (pathname.startsWith('/api/auth/') && pathname !== '/api/auth/register') {
    const ip = getClientIP(request);
    const result = checkRateLimit(ipStore, `auth:${ip}`, AUTH_LIMIT.windowMs, AUTH_LIMIT.maxRequests...
    if (!result.allowed) {
      return NextResponse.json(
        { error: '??????????5?????' },
        {
          status: 429,
          headers: {
            'Retry-After': String(Math.ceil((result.resetAt - Date.now()) / 1000)),
            'X-RateLimit-Remaining': '0',
          },
        },
      );
    }
  }

  // ??? Rate limiting for /api/ai-checkup/* (per user, daily) ???
  if (pathname.startsWith('/api/ai-checkup/')) {
    // User ID from header (set by client after auth), fallback to IP
    const userId = request.headers.get('x-user-id') || getClientIP(request);
    const result = checkRateLimit(userStore, `ai:${userId}`, AI_CHECKUP_LIMIT.windowMs, AI_CHECKUP_L...
    if (!result.allowed) {
      return NextResponse.json(
```

```
        { error: '??AI?????????????' },
        {
          status: 429,
          headers: {
            'Retry-After': String(Math.ceil((result.resetAt - Date.now()) / 1000)),
            'X-RateLimit-Remaining': '0',
            'X-RateLimit-Limit': String(AI_CHECKUP_LIMIT.maxRequests),
          },
        },
      );
    }
    // Add rate limit headers to successful responses
    const response = NextResponse.next();
    response.headers.set('X-RateLimit-Remaining', String(result.remaining));
    response.headers.set('X-RateLimit-Limit', String(AI_CHECKUP_LIMIT.maxRequests));
    return response;
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/api/auth/:path*', '/api/ai-checkup/:path*'],
};

// ========== src/server.ts ==========
import { createServer } from 'http';
import { parse } from 'url';
import next from 'next';

const dev = process.env.COZE_PROJECT_ENV !== 'PROD';
const hostname = process.env.HOSTNAME || 'localhost';
const port = parseInt(process.env.PORT || '5000', 10);

// Security headers applied to all responses
const SECURITY_HEADERS: Record<string, string> = {
  'X-Frame-Options': 'DENY',
  'X-Content-Type-Options': 'nosniff',
  'X-XSS-Protection': '1; mode=block',
  'Referrer-Policy': 'strict-origin-when-cross-origin',
  'Permissions-Policy': 'camera=(), microphone=(), geolocation=()',
};

function getSupabaseHost(): string {
  try {
    const url = process.env.NEXT_PUBLIC_SUPABASE_URL || '';
    return new URL(url).hostname;
  } catch {
    return '*.supabase.co';
  }
```

```
}

function getCspHeader(): string {
  const supabaseHost = getSupabaseHost();
  return [
    "default-src 'self'",
    "script-src 'self' 'unsafe-inline' 'unsafe-eval'",
    "style-src 'self' 'unsafe-inline' https://fonts.googleapis.cn",
    "font-src 'self' https://fonts.gstatic.cn",
    "img-src 'self' data: blob: https:",
    `connect-src 'self' https://${supabaseHost} wss://${supabaseHost}`,
    "frame-ancestors 'none'",
    "base-uri 'self'",
    "form-action 'self'",
  ].join('; ');
}

// Create Next.js app
const app = next({ dev, hostname, port });
const handle = app.getRequestHandler();

app.prepare().then(() => {
  const server = createServer(async (req, res) => {
    try {
      const parsedUrl = parse(req.url!, true);
      // Inject security headers before handling request
      const originalSetHeader = res.setHeader.bind(res);
      let headersInjected = false;
      const injectSecurityHeaders = () => {
        if (headersInjected) return;
        headersInjected = true;
        for (const [key, value] of Object.entries(SECURITY_HEADERS)) {
          try { originalSetHeader(key, value); } catch { /* already sent */ }
        }
        try { originalSetHeader('Content-Security-Policy', getCspHeader()); } catch { /* already sen...
      };

      // Override writeHead to inject headers before first write
      const originalWriteHead = res.writeHead.bind(res);
      res.writeHead = (...args: unknown[]) => {
        injectSecurityHeaders();
        return originalWriteHead(...args as Parameters<typeof originalWriteHead>);
      };

      // Also hook into setHeader to detect when headers are about to be sent
      res.setHeader = (name: string, value: string | string | number) => {
        return originalSetHeader(name, value);
      };

      await handle(req, res, parsedUrl);
```

```
      // Ensure headers are injected even if writeHead wasn't called explicitly
      injectSecurityHeaders();
    } catch (err) {
      console.error('Error occurred handling', req.url, err);
      res.statusCode = 500;
      res.end('Internal server error');
    }
  });
  server.once('error', err => {
    console.error(err);
    process.exit(1);
  });
  server.listen(port, () => {
    console.log(
      `> Server listening at http://${hostname}:${port} as ${
        dev ? 'development' : process.env.COZE_PROJECT_ENV
      }`,
    );
  });
});

// ========== src/storage/database/shared/relations.ts ==========
// Relations definitions have been removed alongside schema.ts table definitions.
// All database operations use Supabase client directly.

// ========== src/storage/database/shared/schema.ts ==========
import { pgTable, text, timestamp, boolean, integer, jsonb, numeric, varchar, uuid, serial, index } ...

// ==================== ????? ====================

export const companies = pgTable('companies', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  name: text('name').notNull(),
  industry: text('industry').default('??'),
  teamSize: integer('team_size').default(1),
  contactName: text('contact_name'),
  contactPhone: text('contact_phone'),
  plan: text('plan').default('basic'),
  planExpiresAt: timestamp('plan_expires_at', { withTimezone: true }),
  aiCreditsRemaining: integer('ai_credits_remaining').default(3),
  serviceLevel: text('service_level').default('self'),
  status: text('status').default('active'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
  planStart: timestamp('plan_start', { withTimezone: true }),
  planEnd: timestamp('plan_end', { withTimezone: true }),
  planPeriod: text('plan_period').default('monthly'),
  brandName: text('brand_name'),
  categories: text('categories').default('[]'),
  priceRange: text('price_range'),
```

```
  platforms: text('platforms').default('[]'),
  dailyConsultations: text('daily_consultations'),
  painPoints: text('pain_points').default('[]'),
  supplyType: text('supply_type'),
  installService: boolean('install_service').default(false),
  returnPolicy: text('return_policy'),
  profileCompleted: boolean('profile_completed').default(false),
  seatLimit: integer('seat_limit').default(1),
  seatUsed: integer('seat_used').default(1),
  trialEndAt: timestamp('trial_end_at', { withTimezone: true }),
});

export const users = pgTable('users', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  email: text('email').notNull(),
  passwordHash: text('password_hash').notNull(),
  displayName: text('display_name'),
  role: text('role').default('agent'),
  userType: text('user_type').default('small'),
  aiCreditsRemaining: integer('ai_credits_remaining').default(3),
  status: text('status').default('active'),
  lastLoginAt: timestamp('last_login_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
  remainingCredits: integer('remaining_credits').default(3),
  industry: varchar('industry', { length: 255 }),
  teamSize: varchar('team_size', { length: 255 }),
  gender: varchar('gender', { length: 50 }).default('??'),
  bio: text('bio').default(''),
  position: varchar('position', { length: 255 }),
  industryProfileCompleted: boolean('industry_profile_completed').default(false),
  expiresAt: timestamp('expires_at', { withTimezone: true }),
});

export const agents = pgTable('agents', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  name: text('name').notNull(),
  employeeId: text('employee_id'),
  hireDate: timestamp('hire_date', { withTimezone: true }),
  position: text('position').default('????'),
  trainingStage: text('training_stage').default('??'),
  status: text('status').default('??'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
  teamId: varchar('team_id', { length: 36 }),
  roleTag: text('role_tag').default('??'),
  skillTags: text('skill_tags').default('{}'),
  userId: uuid('user_id'),
```

```
});

// ==================== KPI ??? ====================

export const kpiSchemes = pgTable('kpi_schemes', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  name: text('name').notNull(),
  positions: jsonb('positions').notNull().default([]),
  cycle: varchar('cycle', { length: 50 }).notNull().default('monthly'),
  scoringSystem: varchar('scoring_system', { length: 50 }).notNull().default('percentage'),
  selectedDimensionIds: jsonb('selected_dimension_ids').notNull().default([]),
  dimensionWeights: jsonb('dimension_weights').notNull().default({}),
  faultTolerance: jsonb('fault_tolerance').notNull().default({}),
  customDimensions: jsonb('custom_dimensions').notNull().default([]),
  customIndicators: jsonb('custom_indicators').notNull().default([]),
  aiEvaluation: jsonb('ai_evaluation'),
  status: varchar('status', { length: 50 }).notNull().default('draft'),
  effectivePeriod: varchar('effective_period', { length: 100 }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).notNull().defaultNow(),
});

export const kpiAssessments = pgTable('kpi_assessments', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  schemeId: varchar('scheme_id', { length: 36 }).notNull(),
  agentId: varchar('agent_id', { length: 36 }).notNull(),
  period: varchar('period', { length: 100 }).notNull(),
  totalScore: numeric('total_score'),
  totalDeduction: numeric('total_deduction').default('0'),
  totalBonus: numeric('total_bonus').default('0'),
  salaryEffect: text('salary_effect'),
  hrAction: text('hr_action'),
  status: varchar('status', { length: 50 }).notNull().default('draft'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).notNull().defaultNow(),
});

export const kpiAssessmentDetails = pgTable('kpi_assessment_details', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  assessmentId: varchar('assessment_id', { length: 36 }).notNull(),
  dimensionId: varchar('dimension_id', { length: 36 }).notNull(),
  indicatorId: varchar('indicator_id', { length: 36 }).notNull(),
  indicatorName: varchar('indicator_name', { length: 255 }).notNull(),
  targetValue: varchar('target_value', { length: 255 }).notNull(),
  actualValue: varchar('actual_value', { length: 255 }),
  isAchieved: boolean('is_achieved'),
  scoreChange: numeric('score_change').default('0'),
  faultToleranceUsed: boolean('fault_tolerance_used').default(false),
```

```
  faultToleranceReason: text('fault_tolerance_reason'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const kpiPlans = pgTable('kpi_plans', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  schemeId: varchar('scheme_id', { length: 36 }).notNull(),
  name: text('name').notNull(),
  period: varchar('period', { length: 100 }).notNull(),
  status: varchar('status', { length: 50 }).default('active'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

export const kpiRecords = pgTable('kpi_records', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  agentId: varchar('agent_id', { length: 36 }).notNull(),
  assessmentId: varchar('assessment_id', { length: 36 }),
  score: numeric('score'),
  period: varchar('period', { length: 100 }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ???? ======================

export const phraseLibrary = pgTable('phrase_library', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }),
  category: varchar('category', { length: 100 }).notNull(),
  content: text('content').notNull(),
  isPreset: boolean('is_preset').default(true),
  useCount: integer('use_count').default(0),
  createdBy: varchar('created_by', { length: 36 }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
  isCase: boolean('is_case').default(false),
  sourceId: varchar('source_id', { length: 36 }),
  scene: text('scene'),
  question: text('question'),
  answer: text('answer'),
  tags: text('tags'),
  reviewStatus: varchar('review_status', { length: 50 }).default('???'),
  reviewNote: text('review_note'),
  reviewedAt: timestamp('reviewed_at', { withTimezone: true }),
  reviewedBy: varchar('reviewed_by', { length: 36 }),
  expiresAt: timestamp('expires_at', { withTimezone: true }),
  freshnessStatus: varchar('freshness_status', { length: 50 }).default('normal'),
});
```

```
// ====================== SOP ??? ======================

export const sopTemplates = pgTable('sop_templates', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }),
  category: text('category').notNull(),
  name: text('name').notNull(),
  scenario: text('scenario'),
  stepsJson: text('steps_json').default('[]'),
  role: text('role').default('????'),
  isPreset: boolean('is_preset').default(false),
  needsUpdate: boolean('needs_update').default(false),
  version: integer('version').default(1),
  updatedBy: text('updated_by'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

// ====================== ??? ======================

export const workOrders = pgTable('work_orders', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }),
  userId: varchar('user_id', { length: 36 }),
  customerName: varchar('customer_name', { length: 255 }),
  customerPhone: varchar('customer_phone', { length: 50 }),
  query: text('query'),
  category: varchar('category', { length: 100 }),
  aiJudgment: text('ai_judgment'),
  aiScript: text('ai_script'),
  priority: varchar('priority', { length: 50 }).default('??'),
  status: varchar('status', { length: 50 }).default('???'),
  result: text('result'),
  sourceType: varchar('source_type', { length: 50 }).default('ai_generate'),
  problemSolutionId: uuid('problem_solution_id'),
  createdAt: timestamp('created_at').defaultNow(),
  completedAt: timestamp('completed_at'),
  updatedAt: timestamp('updated_at').defaultNow(),
  orderNo: varchar('order_no', { length: 50 }),
});

// ====================== ??? ======================

export const courses = pgTable('courses', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }),
  title: text('title').notNull(),
  category: varchar('category', { length: 100 }),
  description: text('description'),
```

```
  content: text('content').default('[]'),
  isPreset: boolean('is_preset').default(true),
  sortOrder: integer('sort_order').default(0),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
  feishuDocUrl: text('feishu_doc_url'),
});

export const courseLessons = pgTable('course_lessons', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  courseId: varchar('course_id', { length: 36 }).notNull(),
  title: text('title').notNull(),
  content: text('content'),
  sortOrder: integer('sort_order').default(0),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const courseHighlights = pgTable('course_highlights', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  courseId: varchar('course_id', { length: 36 }).notNull(),
  title: text('title').notNull(),
  content: text('content'),
  sortOrder: integer('sort_order').default(0),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const learningRecords = pgTable('learning_records', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  courseId: varchar('course_id', { length: 36 }).notNull(),
  lessonId: varchar('lesson_id', { length: 36 }),
  progress: integer('progress').default(0),
  completed: boolean('completed').default(false),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

export const trainingProgress = pgTable('training_progress', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  agentId: varchar('agent_id', { length: 36 }).notNull(),
  stage: text('stage').notNull(),
  progress: integer('progress').default(0),
  status: varchar('status', { length: 50 }).default('in_progress'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

// ====================== ????? ======================
```

```
export const productProfiles = pgTable('product_profiles', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  name: text('name').notNull(),
  category: varchar('category', { length: 100 }),
  specifications: jsonb('specifications'),
  features: jsonb('features'),
  manualUrl: text('manual_url'),
  videoUrl: text('video_url'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

// ==================== AI ??? ====================

export const aiChatHistory = pgTable('ai_chat_history', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  sessionType: varchar('session_type', { length: 50 }),
  question: text('question').notNull(),
  answer: text('answer'),
  feedback: text('feedback'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const aiCheckupSubmissions = pgTable('ai_checkup_submissions', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  checkupType: varchar('checkup_type', { length: 100 }).notNull(),
  input: text('input').notNull(),
  result: text('result'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const healthCheck = pgTable('health_check', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  checkType: varchar('check_type', { length: 100 }).notNull(),
  status: varchar('status', { length: 50 }).default('pending'),
  result: jsonb('result'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ==================== ????? ====================

export const qualityFeedbacks = pgTable('quality_feedbacks', {
```

```
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  targetType: varchar('target_type', { length: 50 }).notNull(),
  targetId: varchar('target_id', { length: 36 }).notNull(),
  rating: integer('rating'),
  feedback: text('feedback'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const qualityInspections = pgTable('quality_inspections', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  agentId: varchar('agent_id', { length: 36 }).notNull(),
  inspectionType: varchar('inspection_type', { length: 100 }).notNull(),
  score: numeric('score'),
  details: jsonb('details'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const chatCheckResults = pgTable('chat_check_results', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  agentId: varchar('agent_id', { length: 36 }).notNull(),
  chatId: varchar('chat_id', { length: 36 }),
  result: jsonb('result'),
  score: numeric('score'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ==================== ??? ====================

export const costRecords = pgTable('cost_records', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  category: varchar('category', { length: 100 }).notNull(),
  amount: numeric('amount').notNull(),
  description: text('description'),
  recordedAt: timestamp('recorded_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const costBaselines = pgTable('cost_baselines', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  category: varchar('category', { length: 100 }).notNull(),
  baselineAmount: numeric('baseline_amount').notNull(),
  alertThreshold: numeric('alert_threshold'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});
```

```
export const costAlertReviews = pgTable('cost_alert_reviews', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  costRecordId: varchar('cost_record_id', { length: 36 }).notNull(),
  reviewedBy: varchar('reviewed_by', { length: 36 }).notNull(),
  status: varchar('status', { length: 50 }).default('pending'),
  note: text('note'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ==================== ??? ====================

export const paymentOrders = pgTable('payment_orders', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  orderNo: varchar('order_no', { length: 50 }).notNull(),
  plan: varchar('plan', { length: 50 }).notNull(),
  amount: numeric('amount').notNull(),
  period: varchar('period', { length: 50 }).notNull(),
  paymentMethod: varchar('payment_method', { length: 50 }),
  screenshotUrl: text('screenshot_url'),
  status: varchar('status', { length: 50 }).default('pending'),
  remark: text('remark'),
  confirmedBy: varchar('confirmed_by', { length: 36 }),
  confirmedAt: timestamp('confirmed_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const subscriptions = pgTable('subscriptions', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  plan: varchar('plan', { length: 50 }).notNull(),
  status: varchar('status', { length: 50 }).default('active'),
  startDate: timestamp('start_date', { withTimezone: true }),
  endDate: timestamp('end_date', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

export const rechargeLogs = pgTable('recharge_logs', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  amount: numeric('amount').notNull(),
  type: varchar('type', { length: 50 }).notNull(),
  description: text('description'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});
```

```
export const businessRecords = pgTable('business_records', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  type: varchar('type', { length: 50 }).notNull(),
  amount: numeric('amount'),
  description: text('description'),
  recordedAt: timestamp('recorded_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const cashFlowRecords = pgTable('cash_flow_records', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  type: varchar('type', { length: 50 }).notNull(),
  amount: numeric('amount').notNull(),
  category: varchar('category', { length: 100 }),
  description: text('description'),
  recordedAt: timestamp('recorded_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const financeRecords = pgTable('finance_records', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  category: varchar('category', { length: 100 }).notNull(),
  amount: numeric('amount').notNull(),
  type: varchar('type', { length: 50 }).notNull(),
  description: text('description'),
  recordedAt: timestamp('recorded_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ==================== ??? ====================

export const teams = pgTable('teams', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  name: text('name').notNull(),
  description: text('description'),
  leaderId: varchar('leader_id', { length: 36 }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

export const invitations = pgTable('invitations', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  email: text('email').notNull(),
  role: varchar('role', { length: 50 }).notNull(),
  invitedBy: varchar('invited_by', { length: 36 }).notNull(),
```

```
  status: varchar('status', { length: 50 }).default('pending'),
  token: varchar('token', { length: 255 }),
  expiresAt: timestamp('expires_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ??? ======================

export const onboardingTasks = pgTable('onboarding_tasks', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  title: text('title').notNull(),
  description: text('description'),
  category: varchar('category', { length: 100 }),
  sortOrder: integer('sort_order').default(0),
  isRequired: boolean('is_required').default(true),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const onboardingRecords = pgTable('onboarding_records', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  agentId: varchar('agent_id', { length: 36 }).notNull(),
  taskId: varchar('task_id', { length: 36 }).notNull(),
  status: varchar('status', { length: 50 }).default('pending'),
  completedAt: timestamp('completed_at', { withTimezone: true }),
  note: text('note'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ??? ======================

export const notifications = pgTable('notifications', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  type: varchar('type', { length: 100 }).notNull(),
  title: text('title').notNull(),
  content: text('content'),
  isRead: boolean('is_read').default(false),
  link: text('link'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ??? ======================

export const schedules = pgTable('schedules', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
```

```
  title: text('title').notNull(),
  description: text('description'),
  startTime: timestamp('start_time', { withTimezone: true }).notNull(),
  endTime: timestamp('end_time', { withTimezone: true }),
  type: varchar('type', { length: 50 }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ???? ======================

export const redemptionCodes = pgTable('redemption_codes', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  code: varchar('code', { length: 100 }).notNull(),
  plan: varchar('plan', { length: 50 }).notNull(),
  period: varchar('period', { length: 50 }).notNull(),
  status: varchar('status', { length: 50 }).default('unused'),
  usedBy: varchar('used_by', { length: 36 }),
  usedAt: timestamp('used_at', { withTimezone: true }),
  expiresAt: timestamp('expires_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ????? ======================

export const customerRecords = pgTable('customer_records', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  agentId: varchar('agent_id', { length: 36 }),
  customerName: varchar('customer_name', { length: 255 }),
  customerPhone: varchar('customer_phone', { length: 50 }),
  customerWechat: varchar('customer_wechat', { length: 100 }),
  query: text('query'),
  result: text('result'),
  category: varchar('category', { length: 100 }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ????? ======================

export const dailyPractice = pgTable('daily_practice', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  question: text('question').notNull(),
  answer: text('answer'),
  isCorrect: boolean('is_correct'),
  score: integer('score'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});
```

```
// ====================== ???? ======================

export const customKnowledge = pgTable('custom_knowledge', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  title: text('title').notNull(),
  content: text('content').notNull(),
  category: varchar('category', { length: 100 }),
  tags: text('tags'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

export const industryKnowledge = pgTable('industry_knowledge', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  title: text('title').notNull(),
  content: text('content').notNull(),
  category: varchar('category', { length: 100 }),
  tags: text('tags'),
  isPreset: boolean('is_preset').default(true),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const knowledgeNotes = pgTable('knowledge_notes', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  knowledgeId: varchar('knowledge_id', { length: 36 }).notNull(),
  note: text('note'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ????? ======================

export const auditLogs = pgTable('audit_logs', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }),
  userId: varchar('user_id', { length: 36 }),
  action: varchar('action', { length: 100 }).notNull(),
  targetType: varchar('target_type', { length: 100 }),
  targetId: varchar('target_id', { length: 36 }),
  details: jsonb('details'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ????? ======================

export const userStats = pgTable('user_stats', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  userId: varchar('user_id', { length: 36 }).notNull(),
```

```
  statType: varchar('stat_type', { length: 100 }).notNull(),
  statValue: varchar('stat_value', { length: 255 }),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

// ====================== ????? ======================

export const activityLogs = pgTable('activity_logs', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }),
  userId: varchar('user_id', { length: 36 }),
  action: varchar('action', { length: 100 }).notNull(),
  details: jsonb('details'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ??? ======================

export const sessions = pgTable('sessions', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  userId: varchar('user_id', { length: 36 }).notNull(),
  token: varchar('token', { length: 255 }).notNull(),
  expiresAt: timestamp('expires_at', { withTimezone: true }).notNull(),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ??? ======================

export const reports = pgTable('reports', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  type: varchar('type', { length: 50 }).notNull(),
  period: varchar('period', { length: 100 }).notNull(),
  data: jsonb('data'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const weeklyReports = pgTable('weekly_reports', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  weekStart: timestamp('week_start', { withTimezone: true }).notNull(),
  weekEnd: timestamp('week_end', { withTimezone: true }).notNull(),
  data: jsonb('data'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const monthlyReports = pgTable('monthly_reports', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
```

```
  month: varchar('month', { length: 7 }).notNull(),
  data: jsonb('data'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const dailyReports = pgTable('daily_reports', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  date: timestamp('date', { withTimezone: true }).notNull(),
  data: jsonb('data'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ====================== ????? ======================

export const profiles = pgTable('profiles', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  userId: varchar('user_id', { length: 36 }).notNull(),
  avatarUrl: text('avatar_url'),
  nickname: text('nickname'),
  preferences: jsonb('preferences'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

export const lessonFeedback = pgTable('lesson_feedback', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  lessonId: varchar('lesson_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  rating: integer('rating'),
  feedback: text('feedback'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const courseProgress = pgTable('course_progress', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  courseId: varchar('course_id', { length: 36 }).notNull(),
  userId: varchar('user_id', { length: 36 }).notNull(),
  progress: integer('progress').default(0),
  completed: boolean('completed').default(false),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

export const managementPlans = pgTable('management_plans', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  title: text('title').notNull(),
  content: text('content'),
  status: varchar('status', { length: 50 }).default('draft'),
```

```
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
});

export const wrongQuestions = pgTable('wrong_questions', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  userId: varchar('user_id', { length: 36 }).notNull(),
  questionId: varchar('question_id', { length: 36 }).notNull(),
  count: integer('count').default(1),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const orders = pgTable('orders', {
  id: varchar('id', { length: 36 }).primaryKey().$defaultFn(() => crypto.randomUUID()),
  companyId: varchar('company_id', { length: 36 }).notNull(),
  orderNo: varchar('order_no', { length: 50 }).notNull(),
  type: varchar('type', { length: 50 }).notNull(),
  amount: numeric('amount').notNull(),
  status: varchar('status', { length: 50 }).default('pending'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

// ?????
export const tables = {
  companies,
  users,
  agents,
  kpiSchemes,
  kpiAssessments,
  kpiAssessmentDetails,
  kpiPlans,
  kpiRecords,
  phraseLibrary,
  sopTemplates,
  workOrders,
  courses,
  courseLessons,
  courseHighlights,
  learningRecords,
  trainingProgress,
  productProfiles,
  aiChatHistory,
  aiCheckupSubmissions,
  healthCheck,
  qualityFeedbacks,
  qualityInspections,
  chatCheckResults,
  costRecords,
  costBaselines,
  costAlertReviews,
```

```
  paymentOrders,
  subscriptions,
  rechargeLogs,
  businessRecords,
  cashFlowRecords,
  financeRecords,
  teams,
  invitations,
  onboardingTasks,
  onboardingRecords,
  notifications,
  schedules,
  redemptionCodes,
  customerRecords,
  dailyPractice,
  customKnowledge,
  industryKnowledge,
  knowledgeNotes,
  auditLogs,
  userStats,
  activityLogs,
  sessions,
  reports,
  weeklyReports,
  monthlyReports,
  dailyReports,
  profiles,
  lessonFeedback,
  courseProgress,
  managementPlans,
  wrongQuestions,
  orders,
};

// ========== src/storage/database/supabase-client.ts ==========
import { createClient, SupabaseClient } from '@supabase/supabase-js';
import { execSync } from 'child_process';
import { getReportBuffer, createWrappedFetch } from 'coze-coding-dev-sdk';

let envLoaded = false;

interface SupabaseCredentials {
  url: string;
  anonKey: string;
}

function loadEnv(): void {
  if (envLoaded || (process.env.COZE_SUPABASE_URL && process.env.COZE_SUPABASE_ANON_KEY)) {
    return;
  }
```

```
  try {
    try {
      require('dotenv').config();
      if (process.env.COZE_SUPABASE_URL && process.env.COZE_SUPABASE_ANON_KEY) {
        envLoaded = true;
        return;
      }
    } catch {
      // dotenv not available
    }

    const pythonCode = `
import os
import sys
try:
    from coze_workload_identity import Client
    client = Client()
    env_vars = client.get_project_env_vars()
    client.close()
    for env_var in env_vars:
        print(f"{env_var.key}={env_var.value}")
except Exception as e:
    print(f"# Error: {e}", file=sys.stderr)
`;

    const output = execSync(`python3 -c '${pythonCode.replace(/'/g, "'\"'\"'")}'`, {
      encoding: 'utf-8',
      timeout: 10000,
      stdio: ['pipe', 'pipe', 'pipe'],
    });

    const lines = output.trim().split('\n');
    for (const line of lines) {
      if (line.startsWith('#')) continue;
      const eqIndex = line.indexOf('=');
      if (eqIndex > 0) {
        const key = line.substring(0, eqIndex);
        let value = line.substring(eqIndex + 1);
        if ((value.startsWith("'") && value.endsWith("'")) ||
            (value.startsWith('"') && value.endsWith('"'))) {
          value = value.slice(1, -1);
        }
        if (!process.env[key]) {
          process.env[key] = value;
        }
      }
    }

    envLoaded = true;
```

```
  } catch {
    // Silently fail
  }
}

/**
 * Check if we are in a build phase where env vars may not be available.
 * During `next build`, routes are imported for page data collection,
 * but Supabase env vars are only injected at runtime.
 */
function isBuildPhase(): boolean {
  // Next.js sets NEXT_PHASE during build
  if (process.env.NEXT_PHASE === 'phase-production-build') return true;
  // If COZE_SUPABASE_URL is missing after loadEnv, assume build phase
  return false;
}

function getSupabaseCredentials(): SupabaseCredentials | null {
  // ??????Coze??????????????????????Supabase
  // TODO: Coze??????????????????
  const OVERRIDE_URL = 'https://ojolpkzgeivgbokotaap.supabase.co';
  const OVERRIDE_ANON_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Im...
  return {
    url: OVERRIDE_URL,
    anonKey: OVERRIDE_ANON_KEY,
  };
}

function getSupabaseServiceRoleKey(): string | undefined {
  // ??????????? Supabase service role key
  return 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Im9qb2xwa3pnZWl2Z2Jva...
}

/**
 * Lazy-initialized Supabase client cache.
 * Key format: "token:<token>" or "no-token" or "service-role"
 */
const clientCache = new Map<string, SupabaseClient>();

function createConfiguredClient(url: string, key: string, globalOptions: Record<string, any>): Supab...
  return createClient(url, key, {
    global: globalOptions,
    db: {
      timeout: 60000,
    },
    auth: {
      autoRefreshToken: false,
      persistSession: false,
    },
  });
```

```
}

function getSupabaseClient(token?: string): SupabaseClient {
  // Only cache the no-token (service role / anon key) client.
  // Token-based clients are per-request and should not be cached.
  if (!token) {
    const cached = clientCache.get('no-token');
    if (cached) return cached;
  }

  const credentials = getSupabaseCredentials();

  // During build phase, env vars may not be available.
  // Return a placeholder client that defers errors to request time.
  if (!credentials) {
    if (isBuildPhase() || !process.env.COZE_SUPABASE_URL) {
      // Create a placeholder client with dummy values so module imports don't crash during build.
      // Actual requests will fail with connection errors, which is expected during build.
      const placeholder = createConfiguredClient('https://placeholder.supabase.co', 'placeholder-key...
      if (!token) clientCache.set('no-token', placeholder);
      return placeholder;
    }
    throw new Error('COZE_SUPABASE_URL is not set');
  }

  const { url, anonKey } = credentials;

  let key: string;
  if (token) {
    key = anonKey;
  } else {
    const serviceRoleKey = getSupabaseServiceRoleKey();
    key = serviceRoleKey ?? anonKey;
  }

  const globalOptions: Record<string, any> = {};
  if (token) {
    globalOptions.headers = { Authorization: `Bearer ${token}` };
  }
  try {
    const buffer = getReportBuffer();
    if (buffer) {
      globalOptions.fetch = createWrappedFetch(buffer, 'supabase');
    }
  } catch {
    // Silent ? reporting setup failure should not block client creation
  }

  const client = createConfiguredClient(url, key, globalOptions);
  if (!token) clientCache.set('no-token', client);
```

```
  return client;
}

export { loadEnv, getSupabaseCredentials, getSupabaseServiceRoleKey, getSupabaseClient };
```